Media Format Allowlist, Decoding Architecture & Privacy Controls

1. Overview and Core Invariants

Case Ace v2.0 accepts externally captured consultations via Route 3: File Import to support advisers who record consultations on managed dictaphones or receive pre-recorded evidence. Because arbitrary media files represent significant attack surfaces and potential privacy leakage vectors, media ingestion is governed by strict technical controls:

Strict Format Allowlist & Default Rejection: Only explicitly justified container formats and audio/video codecs are accepted. All unrecognized or unsupported containers (e.g. MKV, AVI, WMA, RealMedia) are rejected by default.

Sandboxed Worker with Zero Network Access: Decoding runs inside an isolated Web Worker where all network primitives (fetch, XMLHttpRequest, WebSocket, EventSource) are explicitly deleted and neutered.

Immediate Video Frame Discard (Constraint C10): Video containers (MP4, MOV, WebM) have their audio streams extracted and downmixed to 16kHz Float32 mono PCM. Video frames are discarded immediately as decoded—never rendered, previewed, or held in memory.

Zero File Name Invariant: Client file names routinely contain names, dates of birth, and case references. Case Ace reads raw file bytes, immediately dereferences the File object, and never captures, displays, or logs the file name.

Strict Pre-flight Quotas: Files larger than 500 MB or longer than 90 minutes are rejected before decoding to prevent memory exhaustion on 8 GB RAM adviser hardware.

---

2. Media Format Allowlist & Justification

---

3. Rejected Formats & Security Rationales

---

4. Magic-Byte Sniffing Implementation

Case Ace validates file headers using direct byte pattern matching rather than trusting MIME types or extensions:

```
  export function sniffMediaContainer(header: Uint8Array): SniffResult {
    if (header.length < 4) {
      return { isSupported: false, error: 'File too small to identify container signature.' };
    }

    // RIFF -> WAV
    if (header[0] === 0x52 && header[1] === 0x49 && header[2] === 0x46 && header[3] === 0x46) {
      if (header.length >= 12 && header[8] === 0x57 && header[9] === 0x41 && header[10] === 0x56 &&
header[11] === 0x45) {
        return { isSupported: true, container: 'wav', isVideo: false };
      }
    }

    // ID3 or MPEG sync -> MP3
    if (header[0] === 0x49 && header[1] === 0x44 && header[2] === 0x33) {
      return { isSupported: true, container: 'mp3', isVideo: false };
    }
    if (header[0] === 0xFF && (header[1] & 0xE0) === 0xE0) {
      return { isSupported: true, container: 'mp3', isVideo: false };
    }

    // fLaC -> FLAC
    if (header[0] === 0x66 && header[1] === 0x4c && header[2] === 0x61 && header[3] === 0x43) {
      return { isSupported: true, container: 'flac', isVideo: false };
    }

    // OggS -> OGG
    if (header[0] === 0x4f && header[1] === 0x67 && header[2] === 0x67 && header[3] === 0x53) {
      return { isSupported: true, container: 'ogg', isVideo: false };
    }

    // EBML -> WebM
    if (header[0] === 0x1A && header[1] === 0x45 && header[2] === 0xDF && header[3] === 0xA3) {
      return { isSupported: true, container: 'webm', isVideo: true };
```

```
    }

    // ISO Base Media File Format (MP4 / M4A / MOV)
    if (header.length >= 12 && header[4] === 0x66 && header[5] === 0x74 && header[6] === 0x79 &&
header[7] === 0x70) {
      const majorBrand = String.fromCharCode(header[8], header[9], header[10], header[11]);
      if (majorBrand.startsWith('M4A') || majorBrand.startsWith('mp42') ||
majorBrand.startsWith('isom')) {
        return { isSupported: true, container: 'm4a', isVideo: majorBrand.startsWith('isom') ||
majorBrand.startsWith('mp42') };
      }
      if (majorBrand.startsWith('qt  ')) {
        return { isSupported: true, container: 'mov', isVideo: true };
      }
    }

    return { isSupported: false, error: 'Unrecognised container signature. Only WAV, MP3, M4A, AAC,
FLAC, OGG, MP4, MOV, and WebM are permitted.' };
  }
```

5. Sandboxed Web Worker Isolation

To isolate media parser vulnerabilities (e.g., buffer overflows in codec libraries), decoding executes inside client/src/workers/mediaDecoderWorker.ts. The worker sandbox enforces:

Network Primitive Elimination:

```
  delete (self as any).fetch;
  delete (self as any).XMLHttpRequest;
  delete (self as any).WebSocket;
  delete (self as any).EventSource;
```

Memory Transfer Semantics: ArrayBuffer payloads are transferred rather than cloned, zeroing out the main-thread reference upon dispatch.

Fail-Closed Decode Termination: Malformed chunks or corrupted audio frames immediately reject with CORRUPT_MEDIA without attempting recovery or fallback decoders that could trigger undefined behavior.

Clean Worker Lifecycle: The worker instance is explicitly terminated immediately after emitting DECODE_SUCCESS or DECODE_ERROR.